SJSU Student Support Chatbot
Project Workbook

By

Jagruthi Marthala
Lavanya Bandla
Rishith Saginala
Sravan Peethani

03-03-2026

Advisor : Kaikai Liu

1. Literature Search and State of the Art

Research in Artificial Intelligence for higher education support systems has accelerated in three primary areas: Retrieval-Augmented Generation (RAG), conversational AI interfaces, and personalized academic advising and recommendation systems. These domains collectively address the problem of fragmented institutional knowledge and the increasing cognitive load placed on students in large universities.

1. Retrieval-Augmented Generation (RAG)

- Retrieval-Augmented Generation integrates dense retrieval mechanisms with generative language models to produce grounded and verifiable responses. Lewis et al. (2020) introduced RAG as a framework that combines a neural retriever with a sequence-to-sequence generator, allowing responses to be conditioned on externally retrieved documents rather than relying solely on parametric memory [1]. This significantly improves factual consistency and enables citation-based provenance.
- Izacard and Grave (2021) further demonstrated that retrieval-based generative models outperform closed-book models in open-domain question answering tasks by aggregating evidence across multiple passages [2]. Dense retrieval architectures such as ColBERT improve semantic search efficiency and precision in domain-specific corpora [5]. Recent surveys formalize the design space of RAG pipelines and highlight grounding as a primary mitigation strategy for hallucination in large language models [6][7]. These works form the technical foundation of SJSU Copilot's grounded question-answering architecture.

2. Conversational AI for Educational Support

- Conversational AI systems have increasingly been deployed to support student engagement, reduce help-desk load, and provide scalable academic assistance. Winkler and Söllner (2018) highlight that chatbots in educational contexts enhance accessibility and responsiveness, particularly in large institutions where personalized advising resources are limited [3]. Broader research in Educational Data Mining further supports the integration of data-driven personalization into student support systems [8].
- However, existing chatbot systems often lack deep integration with institutional data sources and do not provide citation-backed responses. General-purpose LLM systems offer flexible dialogue but frequently produce unverified or hallucinated outputs without grounding mechanisms [7]. This limits reliability and trustworthiness in academic planning scenarios.

3. Intelligent Academic Planning and Recommendation Systems

- Traditional degree audit systems rely on deterministic rule-based validation to verify prerequisite chains and graduation requirements. While reliable, these systems lack conversational flexibility and contextual reasoning. Modern

recommender systems incorporate hybrid approaches that combine structured rules with embedding-based similarity search [9][10]. Educational recommendation systems similarly leverage personalization techniques to align course suggestions with student profiles and historical performance [11].
- These advances suggest that combining deterministic prerequisite validation with semantic similarity search represents the current state-of-the-art approach for structured academic domains.

4. Information Fragmentation and Academic Overload

- Universities distribute academic policies, course catalogs, deadlines, and advising materials across multiple websites, PDF documents, portals, and email communications. Eppler and Mengis (2004) describe how information overload environments increase cognitive strain and decision errors [4]. In large institutions such as SJSU, students must manually synthesize degree requirements, prerequisite chains, and internship opportunities. This fragmented ecosystem increases the likelihood of missed deadlines and incorrect course planning.
- The literature collectively demonstrates that grounded retrieval, personalization, and conversational interfaces are necessary components for modern academic support systems.

State-of-the-Art Summary

Current solutions address only fragments of the broader student support problem and fail to provide a fully integrated system. Traditional university portals allow students to access academic services and records, but they rely heavily on manual navigation and do not synthesize information across multiple documents or systems. FAQ-based chatbots are restricted to predefined intents and cannot support complex tasks such as multi-step degree planning or prerequisite validation. General-purpose AI systems provide conversational capabilities but lack secure integration with verified institutional data, leading to hallucination risks.

There is currently no unified platform that dynamically retrieves official university documents, grounds responses with citation-backed evidence, personalizes recommendations using authenticated student profiles, tracks degree progress and prerequisite dependencies, and aligns internship opportunities with a student's academic trajectory.

The proposed SJSU Copilot system addresses this gap by integrating retrieval-augmented generation over official SJSU sources, secure student authentication with profile-aware reasoning, deterministic degree validation, hybrid course and internship recommendation engines, and transparent citations for all factual responses. Through this approach, the system transforms fragmented university information into a single intelligent conversational interface that supports students throughout their academic journey.

In contrast to traditional search engines that return isolated links without synthesis, and general-purpose chatbots that generate responses without guaranteed source attribution, emerging systems such as Perplexity AI demonstrate the effectiveness of retrieval-grounded, citation-based conversational search. These systems combine semantic retrieval with

generative reasoning to produce transparent, source-backed answers in real time. However, such architectures have not yet been adapted to secure, authenticated university environments that require deterministic degree validation and personalized academic planning. The SJSU Copilot system extends the Perplexity-style grounded conversational model into the higher education domain by integrating institutional data access, prerequisite validation logic, and student profile-aware recommendation engines within a single secure platform.

References

[1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020.
[2] G. Izacard and E. Grave, “Leveraging Passage Retrieval with Generative Models,” EACL, 2021.
[3] R. Winkler and M. Söllner, “Unleashing the Potential of Chatbots in Education,” 2018.
[4] M. Eppler and J. Mengis, “The Concept of Information Overload,” The Information Society, 2004.
[5] O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT,” SIGIR, 2020.
[6] S. Gao et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv preprint, 2023.
[7] Y. Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys, 2023.
[8] R. S. Baker and K. Yacef, “The State of Educational Data Mining in 2009: A Review and Future Visions,” Journal of Educational Data Mining, 2009.
[9] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” IEEE Computer, 2009.
[10] P. Covington, J. Adams, and E. Sargin, “Deep Neural Networks for YouTube Recommendations,” RecSys, 2016.
[11] X. Chen et al., “A Survey on Recommendation Systems for Education,” IEEE Access, 2020.

2. Project Justification

SJSU Copilot addresses a critical inefficiency within large university ecosystems. Students must navigate academic policies, administrative procedures, and career resources that are distributed across numerous disconnected platforms. This fragmentation reduces clarity, increases decision fatigue, and creates risk of misinformation.

General-purpose AI assistants may provide conversational answers but cannot guarantee accuracy without grounding in official institutional data. In academic environments, unverified responses can negatively impact degree progression and compliance with institutional policies.

The justification for this project is both research-driven and practical.

1. Research Contribution

The system contributes to:

- Applied Retrieval-Augmented Generation architectures
- Hallucination mitigation strategies in institutional domains
- Profile-aware conversational reasoning
- Integration of semantic retrieval with rule-based academic planning

The evaluation metrics (grounding accuracy, reduction in hallucinated responses, recommendation correctness, latency under load) support publication in AI in Education and Human-Centered AI venues.

2. Practical Impact

The system improves:

- Student clarity in academic planning
- Efficiency in identifying graduation requirements
- Awareness of relevant internship opportunities
- Institutional advising scalability

By consolidating verified institutional knowledge into a single intelligent assistant, the platform enhances both usability and trust.

3. Project Requirements

3.1 Essential Features

The system must support several core capabilities that enable accurate academic assistance and personalized student support. First, the platform shall implement Retrieval-Augmented Question Answering, allowing students to ask questions about university policies and receive reliable, citation-backed responses. To achieve this, official SJSU documents will be indexed into a vector database, and for each user query the system will retrieve relevant documents and inject them into the language model so that generated responses are grounded in verified sources and include proper citations.

Second, the system shall support Secure Authentication and Personalization so that responses can adapt to each student's academic profile. Students will authenticate through OAuth2 or the university's single sign-on system, after which relevant profile attributes such as major, completed courses, and credit totals will be retrieved from a secure database. These attributes will be used to tailor responses and recommendations based on the student's academic context.

Third, the platform will provide Degree Progress Tracking, enabling students to understand which graduation requirements remain incomplete. The system will compare a student's completed coursework with official degree requirements, compute remaining requirements, identify prerequisite dependencies, and generate structured summaries of academic progress.

Finally, the system will include a Course Recommendation Engine that suggests courses for upcoming semesters. This component will rank recommended courses by analyzing degree requirements, validating prerequisite chains, and performing semantic matching on course descriptions to ensure that recommendations align with the student's academic trajectory.

3.2 Desired Features

Beyond the essential capabilities, the system may include additional enhancements that improve usability and career readiness. One such feature is Internship and Career Opportunity Alerts, where internship postings are matched to a student's coursework, interests, and skills using semantic similarity and metadata filtering. Relevant opportunities may then be surfaced through the dashboard or chat interface. Another desirable capability is a Citation Transparency Interface, which allows students to preview the sources used to generate responses. The interface will display citation links and snippets of the retrieved passages, improving trust and transparency in the system's outputs.

3.3 Optional Features

Several optional features may be implemented depending on development time and project scope. These include **multi-turn conversational memory** to support more natural dialogue interactions, **an administrative analytics dashboard** for monitoring usage and **system performance**, and **campus navigation integration** to assist students in locating buildings and facilities across campus.

3.4 Non-Functional Requirements

In addition to functional capabilities, the system must meet several non-functional requirements to ensure reliability and usability. The platform should maintain high performance, delivering responses within five seconds under typical load conditions. It must also support scalability, allowing the architecture to expand horizontally through containerized microservices. Security is critical, requiring TLS encryption for data in transit, encrypted storage for sensitive data, and FERPA-aligned handling of student information. The system should achieve high reliability, targeting an uptime of at least 99.9 percent. Finally, the architecture must be maintainable, using a modular design so that retrieval components, recommendation systems, and user interface modules can be updated independently without disrupting the entire platform.

4. Dependencies and Deliverables

System Dependencies (Showstoppers)

The following dependencies are critical. Failure of any of these will halt core system functionality.

1. Institutional Data Access

- The system requires access to official SJSU course catalogs and degree requirement data. Without university approval or structured access, the degree tracking and recommendation engines cannot function.
- **Mitigation:** A fallback ingestion pipeline will parse publicly available PDFs and re-index them using OCR and structured extraction.

2. FERPA Compliance and Legal Approval

- Because the platform handles student academic records, it must comply with FERPA and institutional privacy policies.
- **Mitigation:** All sensitive data will be encrypted using AES-256, transmitted over TLS, and stored within an approved secure cloud environment. No raw academic records will be sent to external LLM APIs.

3. SSO Authentication Integration

- Personalized degree tracking depends on integration with SJSU Single Sign-On.
- **Mitigation**: The system supports OAuth2 integration and a modular authentication layer for secure user mapping.

4. LLM Availability and Cost Constraints

- Conversational reasoning depends on large language models.
- **Mitigation**: A model abstraction layer allows switching between commercial APIs and self-hosted quantized models if costs or availability become issues.

5. Hallucination and Accuracy Risk

- Incorrect graduation advice would undermine trust and usability.
- **Mitigation**: All responses are citation-grounded using RAG, and prerequisite validation is performed using deterministic rule-based logic.

6. Data Drift and Catalog Updates

- University requirements may change.
- **Mitigation**: A scheduled re-indexing pipeline will refresh embeddings and validate document timestamps.

Deliverables

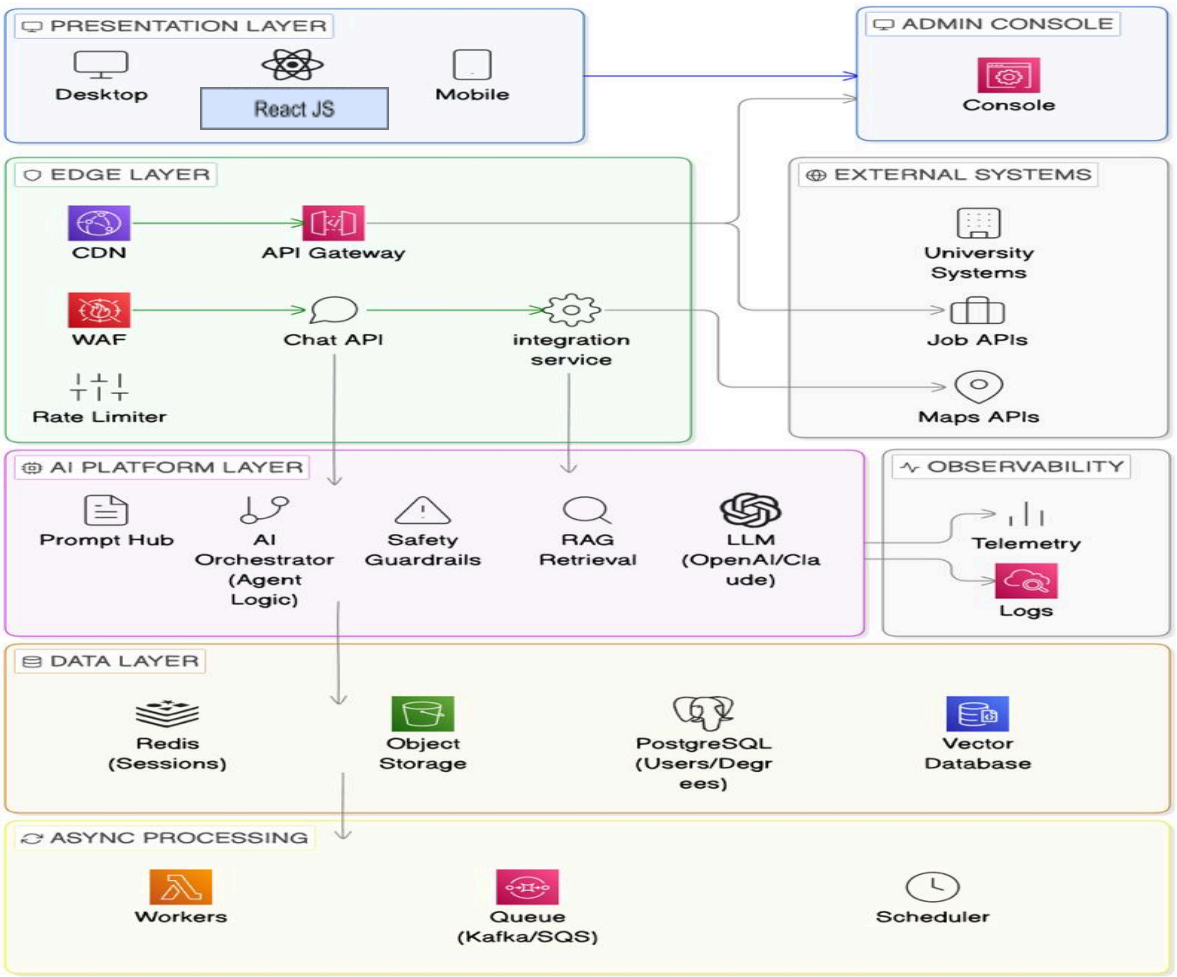
1. Citation-Grounded Academic Chatbot
A RAG-based conversational system that answers questions using official SJSU documents with visible citations.

2. Personalized Degree Progress Engine
A secure system that analyzes completed courses, validates prerequisites, and identifies remaining graduation requirements.

3. Course and Internship Recommendation Engine
A hybrid recommendation system using rule-based validation and embedding similarity, optimized with Redis caching.

4. Research Evaluation Report
A formal study measuring retrieval accuracy, hallucination rate, recommendation precision, and system latency, documented in a publishable research paper.

5. Project Architecture



Architecture Overview:

1. Presentation Layer (Frontend)

- Technology: Built with React / Next.js to provide a fast, responsive chat interface.
- Capabilities: Students can ask academic questions, track degree progress, and view internship alerts.
- Security: Communicates via REST APIs and WebSockets using authenticated user sessions.

2. Edge & Business Logic Layer

- API Gateway: Handles OAuth2/SSO, JWT validation, and request routing to protect backend services.
- Core Services: Modular Python/Node.js services manage degree tracking, course recommendations, and chat orchestration.

3. AI & ML Layer (The "Brain")

- RAG System: Uses FastAPI to perform semantic searches on official SJSU documents for grounded, cited answers.
- Inference: Powered by Llama 2 (13B) to provide human-like advice on university policies and planning.
- Engines: Dedicated modules for degree evaluation and internship matching using semantic similarity.

4. Data Layer (Persistence)

- PostgreSQL: Stores structured data like student profiles, degree records, and chat history.
- Vector Database: Stores document embeddings to enable high-speed RAG retrieval.
- Redis: Manages active user sessions and caches frequently asked questions for speed.
- Object Storage (S3/GCS): Acts as the repository for raw university PDFs and policy catalogs.

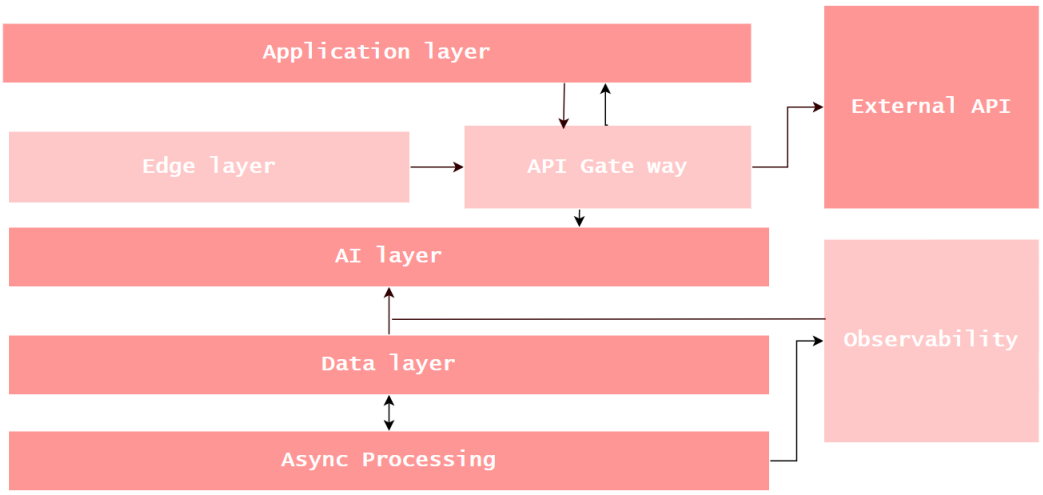
5. Integration & Infrastructure

- Async Processing: Uses RabbitMQ/SQS to handle heavy background tasks like document indexing and embedding.
- Deployment: Containerized with Docker and orchestrated via Kubernetes for horizontal scaling.
- Observability: Monitored using Prometheus, Grafana, and the ELK stack for real-time performance tracking.

6. Project Design

6.1 System Overview

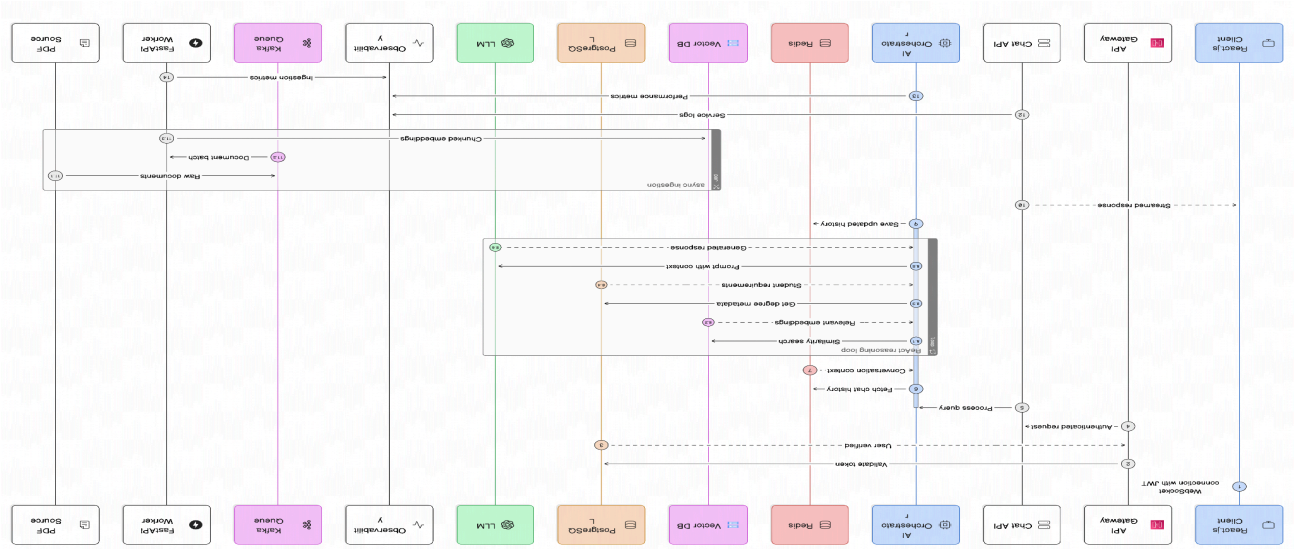
- The application follows a layered system architecture that integrates a frontend interface, backend services, AI processing modules, and data storage systems. The frontend provides a conversational interface where students can ask questions, view degree progress, and receive recommendations.



- User requests are sent to backend services through an API gateway, which handles authentication, request routing, and coordination between system components. AI services process queries using a Retrieval-Augmented Generation (RAG) pipeline and recommendation engines to provide grounded answers and personalized suggestions.

- The system stores structured data such as student profiles and course information in a relational database, while document embeddings are stored in a vector database to enable semantic search and retrieval. This architecture ensures the system is scalable, modular, and capable of integrating additional services in the future.

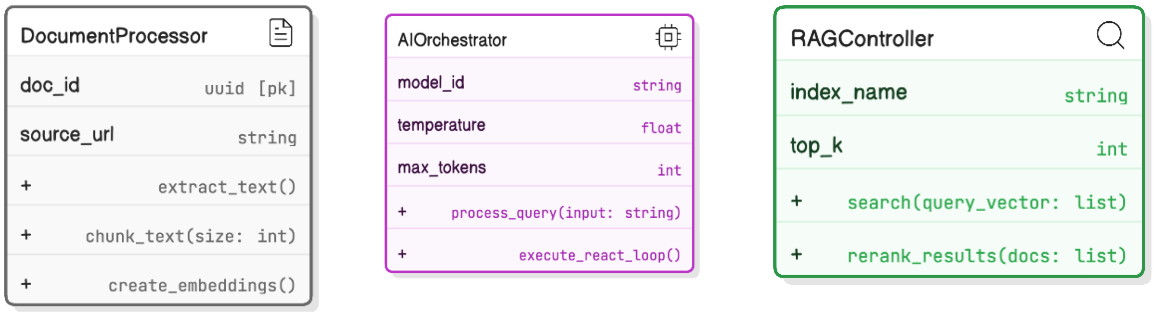
6.2 UML Diagrams



Drive link : uml-diagram.jpeg

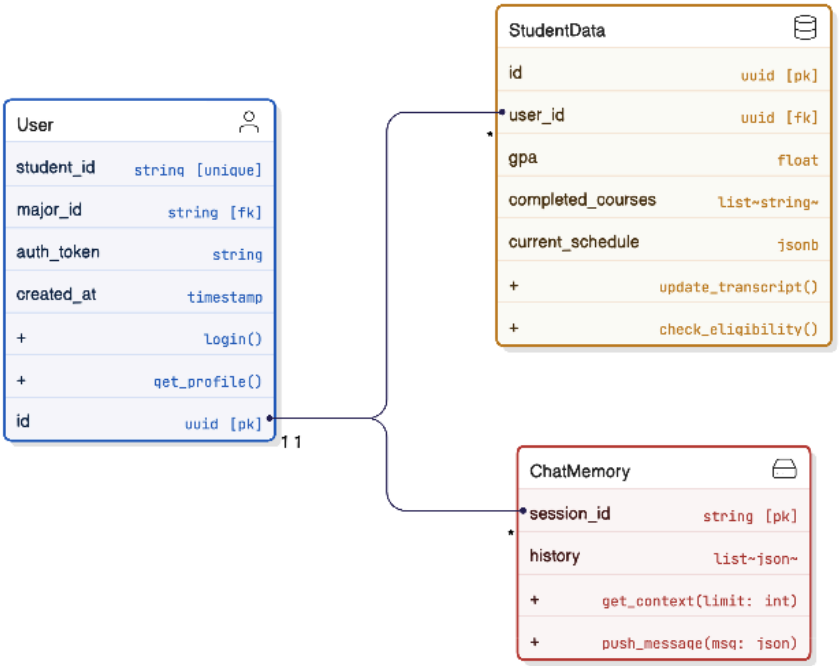
- The sequence diagram illustrates how different system components interact when a user submits a query to SJSU Copilot. The process begins when the Next.js client establishes a WebSocket connection through the API Gateway and sends an authenticated request to the Chat API. The system validates the user's token and verifies authentication before processing the query.
- Once verified, the request is forwarded to the AI Orchestrator, which retrieves the user's chat history and builds the conversation context using data stored in Redis and PostgreSQL. The orchestrator then enters the reasoning workflow, performing a similarity search in the Vector Database to retrieve relevant document embeddings. Additional academic metadata and student requirements may also be fetched from the relational database to support personalized responses.
- The orchestrator constructs a prompt containing the retrieved context and sends it to the LLM for response generation. The generated answer is then returned to the client as a streamed response while the updated conversation history is stored for future interactions. In parallel, an asynchronous ingestion pipeline processes university documents. Raw PDFs are ingested by background workers through Kafka queues, chunked into smaller segments, embedded, and stored in the vector database. Observability tools collect logs and performance metrics to monitor system performance and reliability.

6.3 Class Diagrams



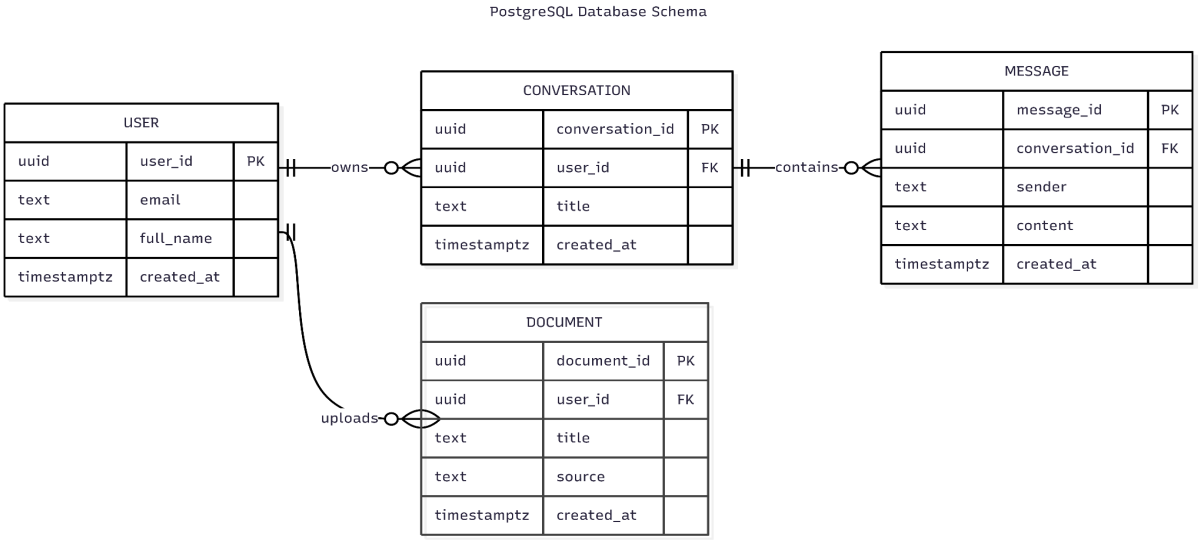
- The UML class diagram represents the main components of the SJSU Copilot system and their responsibilities. The **DocumentProcessor** handles document ingestion by extracting text, chunking content, and generating embeddings for the retrieval system. The **AIOrchestrator** manages language model interactions and processes user queries. The **RAGController** performs vector search and reranks retrieved documents to provide relevant context for response generation.

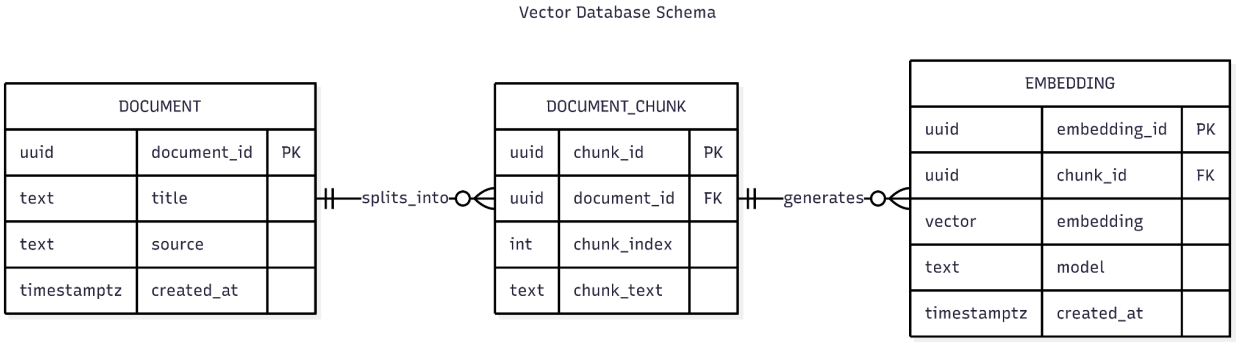
- The **User** class represents authenticated students interacting with the system. It stores profile information and authentication details. The **StudentData** class maintains academic information such as GPA, completed courses, and current schedules, enabling degree progress analysis and personalized recommendations. The **ChatMemory** class stores conversation history to support multi-turn interactions.



- Together, these components enable document retrieval, personalized reasoning, and conversational interaction within the SJSU Copilot platform.

6.4 Database Entity diagrams





- The database schema supports both the conversational system and the document retrieval pipeline used in SJSU Copilot. The PostgreSQL schema manages application data such as users, conversations, and messages. The User table stores student account information, while the Conversation table tracks individual chat sessions associated with each user. Each conversation contains multiple entries in the Message table, which stores the sender, message content, and timestamps. The Document table stores uploaded or indexed documents along with metadata such as title, source, and creation time.

- The Vector Database schema supports the Retrieval-Augmented Generation pipeline. Documents are divided into smaller segments in the Document_Chunk table to enable efficient semantic search. Each chunk is converted into a numerical representation and stored in the Embedding table as vector embeddings generated by an embedding model. These embeddings allow the system to perform similarity search and retrieve relevant document sections when answering user queries.

7. QA, Performance, Deployment Plan

7.1 Testing Plan

The system is validated through a multi-tier testing strategy to ensure reliability and AI safety.

- **Unit & Integration Testing:** Using pytest to validate FastAPI endpoints, JWT authentication, and PostgreSQL CRUD operations for student records. These tests ensure that individual modules function correctly and that backend services communicate reliably with the database and APIs.
- **RAG Evaluation:** Measuring retrieval accuracy (Hit Rate) to verify that the vector database returns the most relevant university policy documents. This evaluation ensures that the RAG pipeline retrieves correct sources before generating responses.
- **AI Guardrail Testing:** Stress-testing the system with adversarial and out-of-scope prompts to detect hallucinations and prevent jailbreak attempts. These tests verify that the assistant responds safely and refuses unsupported or harmful requests.
- **End-to-End Testing:** Simulating a complete student workflow from login to receiving degree planning and internship recommendations. This ensures that the entire system pipeline from frontend interaction to AI response generation operates correctly.

7.2 Performance Evaluation

Performance evaluation measures the system's responsiveness and efficiency under different workloads. Key metrics include response latency for user queries, system throughput, database retrieval time, cache performance, and overall system reliability.

Initial performance targets include processing campus information queries within 5–6 seconds, degree planning queries within 10–12 seconds, and internship searches within approximately 5 seconds, while maintaining an overall system error rate below 1%.

Load testing is conducted by simulating multiple concurrent users interacting with the system to measure response times and evaluate system stability under increased demand.

7.3 Deployment Plan

The SJSU Copilot system is deployed using a containerized architecture with Docker to ensure portability and consistent environments. The frontend application (React), backend services, and AI microservices are packaged as Docker containers.

For the current implementation, the system uses Supabase to host the PostgreSQL database and manage authentication and storage services. The application services are deployed through containerized environments that communicate with Supabase for data access and persistence.

In a production environment, the system can be deployed on **Amazon Web Services (AWS)** to support scalability and reliability. Containerized services can be deployed using Amazon ECS or Kubernetes to manage the backend and AI microservices. The relational database can be hosted using Amazon RDS (PostgreSQL), while document storage can be handled through Amazon S3.

For AI inference, GPU-enabled instances such as **EC2 GPU** instances or **AWS SageMaker** can host the LLM and embedding models used in the RAG pipeline. Additionally, Amazon SQS can manage background processing tasks such as document indexing and internship data updates. This infrastructure enables the system to scale horizontally and handle increased user traffic efficiently.

7.4 Bill of Materials (BoM)

The implementation of SJSU Copilot relies on several software tools and frameworks.

Key components include:

1. **Frontend:** React
2. **Backend & Languages:** Node.js, FastAPI, JavaScript, Python
3. **AI**: Ollama, Embedding Models, FAISS (Vector DB)
4. **Data & Storage:** Supabase PostgreSQL, Redis, Supabase Storage / AWS S3
5. **Infrastructure:** Docker containerization, AWS cloud for production deployment

These technologies provide the infrastructure required to build a scalable and reliable intelligent assistant system.

8. Implementation Plan and Progress

The SJSU Copilot is built using a structured and layered approach, which is in line with the multi-tier architecture (Presentation, API Gateway, Business Logic, ML/AI Engine, Data Layer). The project is being executed in phased iterations to make sure the solution is scalable, modular, and has validation and risk detection. Each phase is designed to take the solution closer to full integration while maintaining a functional vertical slice at every milestone.

1. Environment Setup:

- Programming Environments
  - **Frontend Layer:** React for student dashboard, chat, degree planner, and internship alerts.
  - **API Gateway Layer:** Python + FastAPI for routing, authentication, and orchestration.
  - **AI Engine :** Python + FastAPI for NLP, recommendations, semantic search, and LLM.
  - Database & Storage:
    - PostgreSQL for student data, degree tracking, and course planning.
    - pgvector / FAISS for semantic search and vectors.
    - Redis for caching session data and queries.
    - AWS S3 / GCS for documents, logs, and knowledge base.
  - Cloud & Deployment:
    - Docker for containerization.
    - Kubernetes (future plan) for orchestration and scaling.
    - Target cloud: AWS / GCP.
  - Progress
    - Dockerfiles created for Node.js and FastAPI services (integration testing in progress).
    - PostgreSQL and pgvector set up.
    - GitHub repositories created with a structured directory structure.
    - Development environment set up with local environment

2. Development Tools:

- General Tools
  - Visual Studio Code (Integrated Development Environment)
  - GitHub (version control and project management)
  - Postman (API testing)
  - Docker (containerization)
  - GitHub Actions (planned continuous integration and continuous deployment pipeline)
- AI / ML Tools
  - HuggingFace Transformers (LLM experimentation)
  - spaCy (NLP preprocessing)
  - SentenceTransformers (embedding generation)
  - FAISS/pgvector (semantic search)
  - OpenAI/LLM API (chat and reasoning layer)

- Databases & Services
    - PostgreSQL (student records, degree audits, and internship management)
    - Redis (caching frequently accessed university queries)
    - S3/GCS (document storage and RAG corpus storage)
- Progress
    - Core ML libraries installed and validated.
    - Embedding pipeline validated using sample university catalog data.
    - API Gateway successfully routes to ML Engine.
    - Redis caching validated to speed up repeated queries

3. Simulation & Test Environments

Since SJSU Copilot leverages university data and external APIs,simulation environments were developed for initial testing and verification.

- Mock Data Systems
    - Mocked up a Mock Course Catalog Database for testing semantic search and course recommendations.
    - Developed a Simulated Academic Policy Documents environment for RAG-based question answering.
    - Created a Dummy Internship Listings API for testing recommendations.
    - Developed Sample Student Profile information including major, courses completed, GPA, and preferences.
- External API Mockups
    - LLM API stub for controlled conversational testing.
    - Internship/job API endpoints for testing recommendations.
- Progress
    - RAG pipeline tested with sample academic policy documents.
    - Course semantic search tested using pgvector-based similarity query testing.
    - Internship recommendations tested using internship/job dataset.
    - Staging environment developed for end-to-end API testing.

4.Prototyping Core Modules

1. Authentication & Student Profile Management
    - Implementing JWT-based authentication using FastAPI.
    - Designing the schema for the student profile using PostgreSQL.
    - Degree progress tracking logic prototype complete.
- Progress:
    - Authentication and profile operations are in the phase of implementation.
2. Degree Tracking & Course Planner
    - Implementing a rule-based prerequisite checker.
    - Implementing a course recommendation engine using:
      1.Student course history
      2. Degree major requirements
      3. Embedding similarity search for elective course recommendations.
- Progress:

    - Degree tracking prototype functional with sample data.
      Course similarity search implemented using pgvector.
3. RAG-based Academic Copilot (University Q&A)
    - Context injection implemented to filter hallucinations.
    - Embedding university documents into a vector database.
    - Implementing the Query → Embedding → Similarity Search → Context Injection → LLM Response pipeline.
- Progress:
    - RAG pipeline functional with university policies and course catalog documents.
    - Chat interface connected to ML Engine via API Gateway.
4. Internship & Career Recommendation Engine
    - Hybrid Recommendation:
        - Content-based filtering of skills and job descriptions
        - Semantic Similarity Search via Embeddings
    - Internship APIs Integrated (Mock Phase).
    - Progress:
        - Matching score computed via embedding similarity search Dashboard shows internship recommendations.
5. Conversational AI Layer
    - LLM API integration via FastAPI service
    - Prompt orchestration module:
        - Academic advising mode
        - Degree planning mode
        - Internship guidance mode
    - Progress:
        - Basic chatbot functionality implemented
        - Integration of context memory via Redis session storage in progress

5. Vertical Slice – Prototype of Prototype:

The project successfully implemented a vertical slice that connects all relevant architectural layers.

- **Demonstration Flow**
    - Student logs in via React-based dashboard
    - API Gateway authenticates user request
    - The student asks: “What courses should I take next semester?”
    - Request routed via ML Engine
- **System**
    - Retrieve completed courses from PostgreSQL
    - Validate prerequisites
    - Similarity search for electives
    - Context injected into LLM
- Final recommendation sent back to frontend dashboard
    - Additional Flow – Academic Q&A
    - The student asks: “When is the registration deadline?”
    - RAG pipeline runs vector database search
    - Context injected into LLM

    - Verified response sent back to user
- Progress:
    - End-to-end pipeline fully operational locally
    - RAG + Course Planning fully functional
    - Internship integration partially complete
    - Performance optimization phase initiated

Implementation Roadmap (Phases)

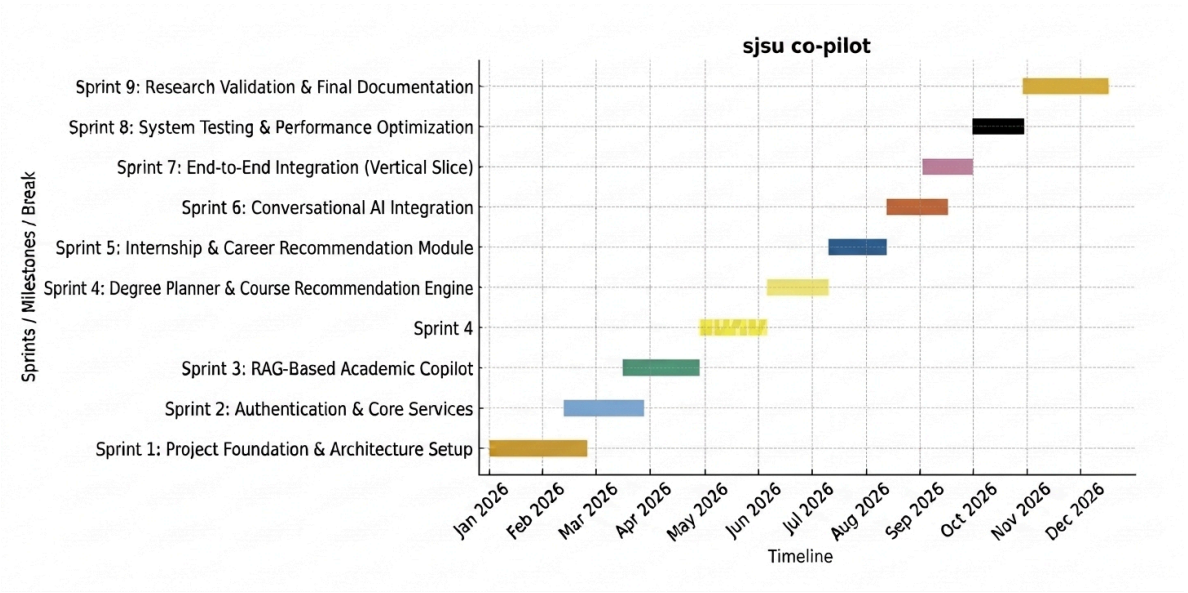
- **Phase 1 (Jan–Feb):** The first phase will involve laying the foundation of the system. This includes setting up the development environment, setting up the GitHub and Docker infrastructures, designing the PostgreSQL database schema, integrating pgvector into the system, and developing the authentication and API Gateway services.
- **Phase 2 (Mar–Apr):** This phase will involve developing the core academic intelligence. This includes developing the degree tracking engine, developing the course planner prototype, integrating the vector database, and developing the RAG-based academic Q&A system with the frontend dashboard.
- **Summer Break (May–July):** This phase will involve refining the documentation, expanding the datasets, reviewing the research, and integrating the feedback from the advisors. There are no new features planned during this phase.
- **Phase 3 (Aug–Sep):** This phase will involve developing the career/internship module. This includes integrating the job APIs, developing the skill extraction and semantic job matching, developing the hybrid recommendation engine, and integrating the internship dashboards with Redis caching.
- **Phase 4 (Oct–Nov):** This phase will involve developing the conversational AI system. This includes integrating the LLM API, developing the multi-turn memory, optimizing the API Gateway, and integrating the entire system.
- **Phase 5 (Dec):** This phase will involve developing the final demo, refining the documentation, and validating the research.

9. Implementation Plan and Progress:

Sprint / Milestone	Timeframe	Tasks
Sprint 1: Project Foundation & Architecture Setup	Jan 2026	Environment setup, GitHub repo initialization, Docker configuration (Rishithh), setup Node.js API Gateway & routing structure (Sravan), configure FastAPI ML engine skeleton & embedding pipeline base (Jagruthi), bootstrap React/Next.js frontend & dashboard layout (Lavanya), design PostgreSQL schema for student profiles & degree tracking.
Sprint 2: Authentication & Core Services	Feb 2026	Implement JWT authentication & role management (Sravan), develop student profile CRUD APIs (Sravan), design database models & Redis caching setup (Rishithh), integrate frontend login/dashboard with backend APIs (Lavanya), test service communication between API Gateway & ML Engine (Jagruthi)
Sprint 3: RAG-Based Academic Copilot	Mar 2026	Collect university catalog & policy documents (Rishithh), build embedding generation using SentenceTransformers & store in pgvector (Jagruthi), implement similarity search APIs (Jagruthi), integrate LLM API for contextual response generation (Sravan), develop chatbot UI prototype (Lavanya), conduct initial evaluation of RAG accuracy.
Summer Break (No Active Sprint)	May – July 2026	Optional documentation updates, research reading, dataset refinement, advisor feedback incorporation
Sprint 4: Degree Planner & Course Recommendation Engine	Aug 2026	Implement prerequisite validation engine (Jagruthi), build rule-based + embedding-based course recommender (Jagruthi), integrate PostgreSQL degree audit system (Rishithh), connect planner APIs through Gateway (Sravan), develop semester planning dashboard & visualization (Lavanya)

Sprint 5: Internship & Career Recommendation Module	Sept 2026	Collect internship/job datasets & define evaluation metrics (Rishithh), implement skill extraction & job matching using embeddings (Jagruthi), integrate internship APIs & caching using Redis (Sravan), develop internship dashboard & recommendation display (Lavanya), benchmark recommendation accuracy
Sprint 6: Conversational AI Integration	Oct 2026	Design prompt orchestration strategies (Jagruthi), integrate OpenAI/LLM API with FastAPI service (Sravan), implement Redis-based multi-turn memory (Rishithh), develop WebSocket chat backend (Sravan), refine chatbot frontend UI & interaction design (Lavanya)
Sprint 7: End-to-End Integration (Vertical Slice)	Nov 2026	Integrate Frontend → API Gateway → ML Engine → Databases (Sravan), optimize ML pipeline & vector search latency (Jagruthi), conduct system integration testing (Rishithh), polish dashboards & unify UI components (Lavanya), prepare demo workflow documentation
Sprint 8: System Testing & Performance Optimization	Early Dec 2026	Conduct load testing & response benchmarking (Rishithh), debug ML model outputs & improve retrieval precision (Jagruthi), improve backend error handling & API stability (Sravan), conduct usability testing & collect student feedback (Lavanya)
Sprint 9: Research Validation & Final Documentation	Late Dec 2026	Design validation methodology & coordinate trials (Rishith), analyze RAG/recommender performance metrics (Jagruthi), document backend scalability & deployment architecture (Sravan), prepare frontend screenshots, UI demos & final presentation slides (Lavanya), compile final report & demo script

Gantt Chart:


sjsu co-pilot
Sprint 9: Research Validation & Final Documentation
Sprint 8: System Testing & Performance Optimization
Sprint 7: End-to-End Integration (Vertical Slice)
Sprint 6: Conversational AI Integration
Sprint 5: Internship & Career Recommendation Module
Sprint 4: Degree Planner & Course Recommendation Engine
Sprint 4
Sprint 3: RAG-Based Academic Copilot
Sprint 2: Authentication & Core Services
Sprint 1: Project Foundation & Architecture Setup
Sprints / Milestones / Break
Jan 2026
Feb 2026
Mar 2026
Apr 2026
May 2026
Jun 2026
Jul 2026
Aug 2026
Sep 2026
Oct 2026
Nov 2026
Dec 2026
Timeline